

Time Series Forecasting using Recurrent Neural Networks

Willie Loftie-Eaton

Computer Science Division, Stellenbosch University

Stellenbosch, South Africa

Email: 24717274@sun.ac.za / willieloea@gmail.com

Index Terms—Recurrent Neural Networks

I. INTRODUCTION

Time-series forecasting uses past observations to predict future values. Recurrent neural networks (RNNs) approach this task by carrying information between sequence positions, allowing predictions to depend on earlier observations. This study compares Elman, Jordan and multi-recurrent networks to investigate how different feedback structures affect forecasting accuracy.

The comparison uses five datasets: airline passengers, minimum temperatures, smart-home sensors, sunspots and a synthetic autoregressive series. Together, they provide different patterns of temporal dependence. A shared data-processing and training procedure supports a consistent comparison, with chronological validation used to select model settings and separate test periods used for final evaluation. A simple persistence forecast provides a reference against which to assess the value of the networks.

The results show that no architecture performs best in every setting, and that recurrent models do not always outperform persistence (Table IV). The report introduces the relevant background, describes the implementation and datasets, and discusses these findings alongside the limitations of the experiments.

II. BACKGROUND

This section introduces neural networks, recurrent feedback and time-series validation, providing the concepts used in the implementation and evaluation.

A. Neural networks

An artificial neural network learns a mapping from input features to outputs using interconnected processing units called neurons. In a feedforward network, information passes from an input layer through one or more hidden layers to an output layer. Each processing neuron forms a weighted sum of its inputs, adds a bias, and applies an activation function [1, Sec. 8.2]:

$$a = \phi \left(\sum_{i=1}^m w_i x_i + b \right), \quad (1)$$

where x_i are the inputs, w_i are connection weights, b is the bias, and ϕ is the activation function. Nonlinear hidden-layer activations allow the network to represent nonlinear relationships.

During supervised training, a forward pass produces a prediction, which is compared with the known target using a loss function. Backpropagation applies the chain rule to calculate the loss gradients with respect to the weights and biases. An optimisation algorithm, such as gradient descent, then uses these gradients to adjust the parameters. Repeating this process over training examples allows the network to learn the input-output relationship [1, Sec. 8.3]. The output layer depends on the task: numerical forecasting can use a single output unit with a linear activation.

B. Recurrent neural networks

Recurrent neural networks (RNNs) process ordered sequences using feedback connections that carry information between time steps. The same weights are reused at each step, while the network's state changes as new observations are processed. Consequently, a prediction can depend on earlier observations as well as the current input. This state acts as a learned summary of the sequence, rather than an explicit copy of every previous observation [1].

In an *Elman RNN* [2], context units retain the previous hidden-layer activations. These are supplied through weighted connections to the hidden layer alongside the current input. A simple state update is

$$\mathbf{h}_t = \phi(W_x \mathbf{x}_t + W_h \mathbf{h}_{t-1} + \mathbf{b}_h), \quad (2)$$

where $\mathbf{x}_t$ is the current input and $\mathbf{h}_{t-1}$ is the previous hidden state. The matrices W_x and W_h and bias $\mathbf{b}_h$ are learned parameters. Because the previous state depends on earlier states, information can influence predictions beyond a single time step.

A *Jordan RNN* [3] instead feeds output activations back through context units to the hidden layer. Some Jordan formulations also include self-connections in the context units, allowing older context to persist.

A combined *multi-recurrent* architecture can incorporate both hidden-state and output feedback. These feedback signals come from earlier time steps.

RNNs can be trained using backpropagation through time, which follows the dependencies across the sequence to calculate gradients. Gradients may shrink or grow excessively over long sequences, making long-term dependencies difficult to learn.

C. Cross-validation for time series data

Cross-validation estimates how well a model generalises to unseen observations and supports the selection of models and hyperparameters. Standard shuffled k -fold cross-validation is unsuitable for forecasting because it can place future observations in the training set while evaluating predictions for the past. Time-series validation instead preserves chronological order: each model is trained on earlier observations and evaluated on a subsequent period.

In walk-forward validation, the evaluation period moves forward between folds. An *expanding window* retains all preceding training observations, whereas a *sliding window* retains a fixed amount of recent history and discards older observations. Expanding windows make use of increasing amounts of data; sliding windows can be useful when older patterns become less representative. Errors across successive validation periods provide a broader assessment than a single split, and the forecast horizon should match the intended prediction task.

A *chronological holdout* reserves a final period for evaluation after model selection. Although this is not itself cross-validation, it complements the repeated validation used for tuning. The final holdout must not influence hyperparameter selection.

III. METHODOLOGY AND IMPLEMENTATION

This section describes the five datasets used, the data pipeline, the three recurrent architectures, and the training, model-selection and evaluation procedures.

The implementation separates data preparation, recurrent models, training and evaluation into shared Python modules. NumPy and pandas provide the data pipeline, and PyTorch implements the networks and automatic differentiation. All three architectures use the same eligible examples, scaling rules, loss function and evaluation procedure. Their feedback connections and validation-selected hidden sizes differ.

A. Dataset Overview

The following five datasets are used throughout the report. These short names identify the forecasting tasks; Section IV-A provides their sources, plots and more detailed discussion of their temporal properties.

Air passengers: Monthly international airline passenger totals, measured in thousands. The task is to predict the next month's total from previous totals in a series with growth and annual seasonality.

Minimum temperatures: Daily minimum temperatures in Melbourne, measured in degrees Celsius. The task is to predict the next day's minimum from past temperatures, which reflect annual seasonality and daily variation.

Smart home: Environmental sensor measurements from a monitored house, recorded at 15-minute intervals. Indoor and outdoor sensor histories are used to predict dining-room and bedroom temperatures at the next interval.

Sunspots: Monthly mean total sunspot numbers, used as an index of solar activity. The task is to predict the next month's value from previous values in a series with approximately 11-year cycles.

AR(2): A synthetic second-order autoregressive series in which each value depends on its two predecessors and added random noise. It provides a controlled stationary forecasting task with known generating dynamics.

B. Forecasting Task and Data Preparation

Let $\mathbf{x}_t \in \mathbb{R}^d$ contain the input features available at forecast origin t , and let $\mathbf{y}_{t+q} \in \mathbb{R}^m$ contain the targets q steps ahead. An example consists of

$$\mathbf{X}_t = (\mathbf{x}_{t-L+1}, \dots, \mathbf{x}_t), \quad \text{target} = \mathbf{y}_{t+q}, \quad (3)$$

where L is the lookahead. A minibatch is a group of B training examples processed together to compute one weight update. Its input array has shape $B \times L \times d$, and its predictions have shape $B \times m$. The pipeline supports configurable inputs, targets and horizons, but every reported experiment uses $q = 1$. Passenger, minimum-temperature, sunspot and AR(2) experiments use the target's own history ($d = m = 1$). Smart-home experiments use dining-room and bedroom temperature, outdoor temperature and outdoor humidity ($d = 4$), predicting both indoor temperatures jointly ($m = 2$).

Dataset-specific scripts prepare regular time indices before window construction. Passenger values require no imputation. The minimum-temperature cleaner removes question-mark prefixes from three numeric entries, restores two missing dates and forward-fills them. Smart-home recordings are processed separately: two missing 15-minute records are inserted and forward-filled, but a long gap between recordings is preserved. Observation flags are assigned before filling so that inserted values can serve as input history without becoming ground-truth targets. The sunspot cleaner preserves the supplied target series, including a historically interpolated February 1824 value, while retaining quality metadata separately from model inputs. AR(2) samples are generated directly with no missing values; the generator settings are given in Section IV-A.

The shared loader validates timestamps and selected numeric features and assigns a segment identifier to each uninterrupted recording. Any example whose input-to-target interval crosses a segment boundary is rejected. Examples with no observed target are also rejected. Validation and test windows may use preceding training observations as context, since those observations are available at the forecast origin.

Each input and target feature is standardized separately:

$$z_{t,j} = \frac{x_{t,j} - \mu_j}{s_j}. \quad (4)$$

The mean μ_j and population standard deviation s_j are fitted using observed rows in the current training interval only, before overlapping windows are constructed. Thus repeated appearances of a row in different windows do not change its scaling weight. Constant features use $s_j = 1$. The same fitted transformation is applied to the corresponding validation or test interval. No differencing, logarithmic transformation or seasonal adjustment is used, and predictions are inverse-transformed before reporting errors in original units.

C. Recurrent Network Implementations

Each network has one hidden layer of H tanh units and a linear output layer. Within an input window, let k index the sequence position, $\mathbf{h}_k \in \mathbb{R}^H$ the hidden activation, and $\mathbf{o}_k \in \mathbb{R}^m$ the output. The Elman network uses hidden-state feedback:

$$\mathbf{h}_k = \tanh(W_x \mathbf{x}_k + W_h \mathbf{h}_{k-1} + \mathbf{b}_h). \quad (5)$$

The Jordan network uses output feedback:

$$\mathbf{h}_k = \tanh(W_x \mathbf{x}_k + W_o \mathbf{o}_{k-1} + \mathbf{b}_h). \quad (6)$$

The multi-recurrent network combines both paths:

$$\mathbf{h}_k = \tanh(W_x \mathbf{x}_k + W_h \mathbf{h}_{k-1} + W_o \mathbf{o}_{k-1} + \mathbf{b}_h). \quad (7)$$

All three compute

$$\mathbf{o}_k = W_y \mathbf{h}_k + \mathbf{b}_y, \quad \hat{\mathbf{y}}_{t+1} = \mathbf{o}_L. \quad (8)$$

Here W_x is $H \times d$, W_h is $H \times H$, W_o is $H \times m$, and W_y is $m \times H$. Each implementation has one hidden bias and one output bias; feedback projections have no additional biases. The output layer is linear because the task is regression on standardized real-valued targets.

Hidden and output contexts are initialized to zero independently for each window. Context stores a one-step copy; no separate leaky context self-connection is used. Jordan and multi-recurrent feedback always uses the network's own intermediate output, not a supplied ground-truth output. Only the last output is directly supervised. Intermediate outputs and hidden states remain attached to the computation graph, allowing gradients to propagate through all L steps. Xavier-uniform initialization [4] is used for the input, output and output-feedback matrices, orthogonal initialization [5] for hidden-feedback matrices, and zero initialization for biases.

D. Loss and Optimization

For a batch of standardized targets, let M_{ij} be one when target j of example i is observed and zero otherwise. The shared loss is masked mean squared error:

$$\mathcal{L} = \frac{\sum_{i=1}^B \sum_{j=1}^m M_{ij} (\hat{z}_{ij} - z_{ij})^2}{\sum_{i=1}^B \sum_{j=1}^m M_{ij}}. \quad (9)$$

In code, observed entries are selected before subtraction to prevent masked placeholders from contaminating the calculation. This gives each observed standardized target entry equal weight.

Gradients are computed by backpropagation through time using PyTorch autograd. Adam updates the weights with learning rate 0.001 and its default moment parameters. Batches contain up to 32 examples, and the total gradient norm is clipped to 1.0 before each update. The search allows at most 100 epochs. There is no dropout or weight decay. Examples are shuffled within the training partition at each epoch, while the time order inside each window is preserved. This does not randomly mix training and validation periods: the partitions remain chronological, and recurrent context is reset between windows.

E. Chronological Validation and Model Selection

The last block of each dataset is reserved as a final test set before tuning. The earlier development interval is evaluated using expanding-window cross-validation: the training prefix grows, and each following validation block is scored once. The step between folds equals the validation-block length. Table I gives the actual row counts and input windows. Only complete validation blocks are emitted; any development rows left after the last complete block are still available for final training. Sliding-window validation is not used in the reported experiments.

Training duration is chosen inside each outer training fold, without consulting its outer validation targets. The final 15% of that training interval, rounded up to whole rows, forms an inner early-stopping block. Separate scalars are fitted on the inner fitting prefix. After each epoch, the inner stopping loss is evaluated; an improvement must exceed 10^{-5} in standardized MSE. Training ends after ten consecutive epochs without such improvement, or at 100 epochs, and the epoch of the best qualifying improvement is retained.

The inner model is then discarded. A new model and optimizer are initialized with the same seed, scalars are fitted on the entire outer training interval, and the model is trained for the retained number of epochs. Only this refitted model is scored on the outer validation block. This separates the choice of training duration from the validation score used to compare hidden sizes.

For each architecture, hidden sizes $H \in \{8, 16, 32\}$ are compared using seed 42. A fold's selection score is the mean of its per-target root mean square errors (RMSEs) in training-standardized units; fold scores are then averaged with equal weight. This avoids letting target units determine the multivariate ranking. It differs from pooling all residuals before computing a single RMSE. Each architecture retains its own lowest-scoring hidden size, with exact ties resolved in favour of the smaller size. Across the five datasets, this produces 207 outer trials, each including inner selection and outer refitting. Early stopping and the size search constrain fitting, but do not guarantee the absence of underfitting or overfitting.

F. Final Evaluation and Reproducibility

For each selected architecture and hidden size, the final epoch count is the integer part of the median selected epoch across its cross-validation (CV) folds, with a minimum of one. New models are fitted on the complete development interval using seeds 11, 22 and 33 and development-only scalars. No test loss is used for early stopping or model selection. This gives 45 final fits. Predictions are made one step ahead using the latest available observations; the evaluation does not recursively substitute model predictions for subsequently observed inputs.

For target j , errors in original units are summarized over

TABLE I

PREPROCESSING AND TEMPORAL SPLIT SETTINGS. ALL SIZES ARE ROW COUNTS, NOT NUMBERS OF CONSTRUCTED EXAMPLES. THE FORECAST HORIZON IS ONE STEP THROUGHOUT.

Dataset	Lookback	Initial training	Validation block	Reserved test	CV folds
Air passengers	12	72	12	24	4
Minimum temperatures	30	1461	365	730	4
Smart home	96	1344	288	672	7
Sunspots	132	2000	264	264	4
AR(2)	2	2000	500	1000	4

the observed test indices $\mathcal{O}_j$:

$$\text{MAE}_j = \frac{1}{|\mathcal{O}_j|} \sum_{i \in \mathcal{O}_j} |\hat{y}_{ij} - y_{ij}|, \quad (10)$$

$$\text{RMSE}_j = \sqrt{\frac{1}{|\mathcal{O}_j|} \sum_{i \in \mathcal{O}_j} (\hat{y}_{ij} - y_{ij})^2}. \quad (11)$$

(MAE stands for mean absolute error.) Persistence predicts each target's latest available value, $\hat{\mathbf{y}}_{t+1} = \mathbf{y}_t$, and is evaluated on the same eligible targets. Metrics are calculated separately for each final fit and reported as their mean and sample standard deviation across seeds, rather than as the performance of an ensemble. Both smart-home outputs are reported separately.

The runner saves configurations, data and implementation hashes, software versions, fitted scalars, model checkpoints, epoch histories and timestamped predictions. These artifacts support resumption and checking of experiment consistency. The results-generation script recomputes errors from all saved observed predictions and verifies identical evaluation rows across architectures and seeds before creating report tables and figures. Automated checks also cover temporal splitting, preprocessing leakage, masking, recurrent equations, gradient propagation and independent window state. These checks establish implementation consistency.

IV. EMPIRICAL PROCEDURE

This section presents the five datasets, examines their temporal patterns and stationarity, and explains the construction of the synthetic AR(2) series.

A. Datasets and Stationarity Analysis

To comprehensively evaluate and compare the Elman, Jordan, and Multi-recurrent neural network architectures, five time series benchmark problems are selected. These datasets embody distinct dynamic regimes, including deterministic trends, multi-frequency seasonality, short-term physical persistence, decadal cycles, and stationary linear stochastic dependence. In this section, we analyze the four empirical time series benchmarks, examine their visual patterns, and formally evaluate their stationarity properties using rolling statistics, the Autocorrelation Function (ACF), the Augmented Dickey-Fuller (ADF) unit-root test [6], and the Kwiatkowski-Phillips-Schmidt-Shin (KPSS) stationarity test [7]. A synthetic second-order autoregressive process, AR(2), is selected as the fifth dataset.

A weakly stationary time series possesses a time-invariant mean $\mathbb{E}[y_t] = \mu$, a constant variance $\text{Var}(y_t) = \sigma^2 < \infty$,

and an autocovariance $\text{Cov}(y_t, y_{t-k}) = \gamma_k$ that depends strictly on the temporal lag k rather than the absolute calendar time t . Table II reports descriptive properties and statistical test outcomes, while Fig. 1 depicts the observed trajectories alongside rolling central tendencies and volatility bands.

1) *International Airline Passengers*: The monthly airline passengers dataset [8] consists of 144 consecutive monthly totals (January 1949 to December 1960) measured in thousands of passengers. As illustrated in Fig. 1(a), the series displays a clear secular upward trend combined with multiplicative annual seasonality whose oscillation amplitude increases proportionally with the series level. The autocorrelations in Table II demonstrate extremely strong persistence at lag 1 ($\text{ACF}(1) = 0.948$) and substantial seasonal correlation at the 12-month period ($\text{ACF}(12) = 0.760$). The ADF test fails to reject the null hypothesis of a stochastic unit root ($t = 0.82, p = 0.992$), whereas the KPSS test decisively rejects the null hypothesis of level stationarity ($kpss = 1.65, p < 0.01$). Consequently, the raw series is classified as **non-stationary** in both its mean and variance.

2) *Melbourne Daily Minimum Temperatures*: The Melbourne temperature dataset records 3,652 daily observations from January 1, 1981 through December 31, 1990. The target is daily minimum temperature in degrees Celsius. As shown in Fig. 1(b), the series is dominated by an annual weather cycle ($s = 365$ days) exhibiting strong seasonal autocorrelation ($\text{ACF}(365) = 0.480$) combined with substantial short-term day-to-day persistence ($\text{ACF}(1) = 0.774$). In Table II, the standard ADF test rejects the unit-root null ($t = -4.44, p < 0.001$) and KPSS fails to reject level stationarity on an aggregated decadal basis ($kpss = 0.06, p = 0.100$). Nevertheless, the series has an explicitly time-dependent seasonal mean function $\mathbb{E}[y_t] = \mu(t)$ across the calendar year. Because weak stationarity requires unconditional invariance of the mean over time, this annual cyclicity renders the series **seasonally non-stationary**.

3) *Smart-Home Sensors (SML2010)*: The SML2010 dataset [9] comprises high-frequency environmental sensor telemetry sampled every 15 minutes within a domestic smart house. Dining-room temperature (variable 2, in $^{\circ}\text{C}$) is selected as the primary forecasting target. The dataset spans two non-contiguous recording intervals (March 13 to May 2, 2012) separated by a 6-day unrecorded gap. Fig. 1(c) highlights strong 24-hour oscillations ($s = 96$ time steps, $\text{ACF}(96) = 0.742$) resulting from solar radiation and domestic heating, overlaid upon shifting baseline temperature trends across weeks. Evaluated on the continuous first recording segment ($N = 2,764$), the KPSS result in Table II firmly

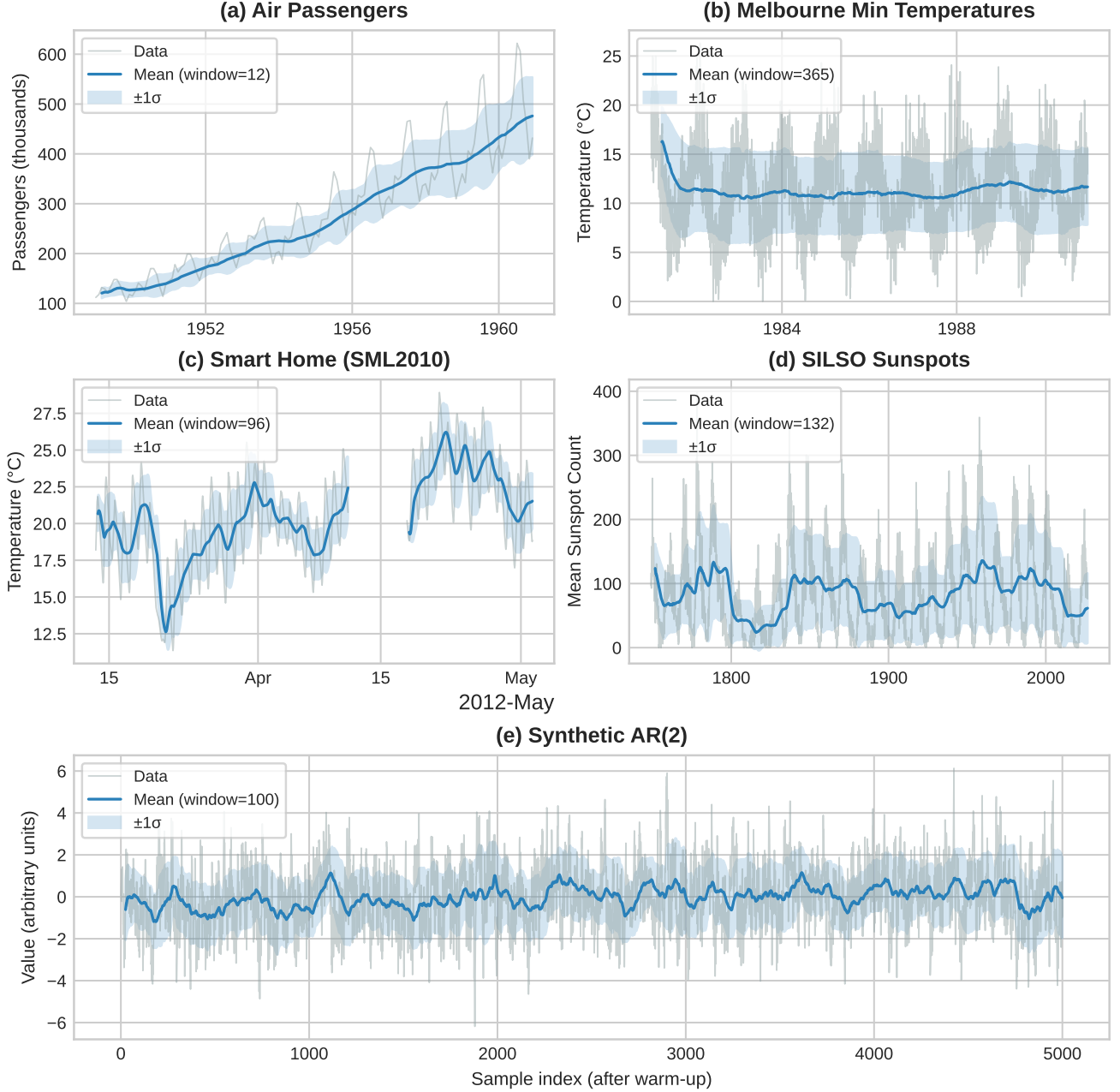


Fig. 1. Target feature trajectories for all five benchmark datasets: (a) International Airline Passengers with 12-month rolling mean and $\pm 1\sigma$ band; (b) Melbourne Daily Minimum Temperatures with 365-day rolling statistics; (c) Smart Home (SML2010) dining-room temperature with 24-hour (96-step) rolling statistics across both recording segments; (d) SILSO Monthly Sunspot Numbers with 132-month (approximately 11-year) rolling statistics; (e) synthetic AR(2) with 100-sample rolling statistics, indexed after discarding warm-up samples. Shaded bands show rolling mean ± 1 rolling standard deviation, not confidence intervals. The AR(2) smoothing window does not represent a seasonal period.

rejects stationarity ($kpss = 1.17, p < 0.01$). The series is classified as **non-stationary** due to periodicity and shifting baseline temperature levels.

4) *SILSO Monthly Mean Sunspot Numbers*: The SILSO Version 2.0 monthly sunspot series [10] spans 3,332 monthly records from January 1749 through August 2026. The target is the monthly mean total sunspot count. Fig. 1(d) shows the quasi-periodic Schwabe solar cycle with a nominal period of approximately 11 years ($s \approx 132$ months, $ACF(132) = 0.553$). While the ADF test in Table II rejects a unit root ($t = -10.71, p < 0.001$) reflecting global mean-reversion

toward long-term historical averages, the solar cycles exhibit non-constant amplitude, asymmetrical cycle shapes, and multi-decadal amplitude modulations. Stationarity is therefore classified as **uncertain and cyclical**, demanding models capable of capturing long temporal memory without assuming rigid harmonic periodicity.

5) *Synthetic AR(2) Process*: The fifth dataset is generated using

$$x_t = 0.6x_{t-1} + 0.2x_{t-2} + \epsilon_t, \quad \epsilon_t \stackrel{\text{iid}}{\sim} \mathcal{N}(0, 1). \quad (12)$$

TABLE II
EMPIRICAL STATIONARITY AND DESCRIPTIVE DIAGNOSTICS FOR THE FOUR TIME SERIES DATASETS

Dataset	Obs (N)	Mean $\pm$ Std	ACF(1)	ACF(Cycle)	ADF (p)	KPSS (p)	Primary Evidence
Air Passengers	144	280.3 $\pm$ 119.5	0.948	0.760 (12)	0.82 (0.992)	1.65 (0.010)	Non-stationary (trend and growing variance); ADF $p > 0.99$, KPSS $p < 0.01$.
Melbourne Min Temps	3,652	11.2 $\pm$ 4.1	0.774	0.480 (365)	-4.44 (0.000)	0.06 (0.100)	Non-stationary (annual cycle $s = 365$, ACF(365) = 0.48); time-varying mean.
Smart Home (Dining)	2,764	19.2 $\pm$ 2.9	0.999	0.742 (96)	-5.34 (0.000)	1.17 (0.010)	Non-stationary (diurnal cycle $s = 96$, ACF(96) = 0.74); level shifts, KPSS $p < 0.01$.
SILSO Sunspots	3,332	82.2 $\pm$ 67.6	0.918	0.553 (132)	-10.71 (0.000)	0.11 (0.100)	Cyclical / Mixed (solar cycle $s \approx 132$, ACF(132) = 0.55); decadal amplitude modulation.

Autoregressive processes predict a value from its preceding values [11]. The chosen coefficients define a stationary solution because both roots of $1 - 0.6z - 0.2z^2$ lie outside the unit circle. Its theoretical mean is zero and variance is 50/21. The generator starts from two zeros, discards 1,000 warm-up steps to suppress initialization effects, and retains 5,000 samples using NumPy’s PCG64 generator with seed 2026. Artificial daily timestamps provide an ordered index only. No imputation or differencing is applied. Figure 1(e) shows the retained realization fluctuating around zero, with a 100-sample rolling mean and standard-deviation band. This visual behaviour is consistent with the stationary construction, although a finite plot alone does not prove stationarity.

The preprocessing preset uses a two-step input window and one-step forecast horizon, reserves the final 1,000 samples for testing, and provides four 500-sample validation blocks after an initial 2,000-sample training block. Scaling is fitted separately on each training fold. These are experimental design choices, not a standardized benchmark configuration. The series provides a controlled test of short linear dependence. The known conditional-mean forecast, $\hat{x}_{t+1} = 0.6x_t + 0.2x_{t-1}$, has population mean squared error 1 and supplies a reference for future comparison alongside persistence. The reported AR(2) results compare the RNNs with persistence.

V. RESULTS AND DISCUSSION

This section compares the final forecasting results with persistence, discusses the findings for each dataset, and considers the limitations of the comparison.

Table III records the model settings selected on development data. Each selected model was fitted afresh using initialization seeds 11, 22 and 33 and evaluated on the reserved chronological test period. Table IV reports mean absolute error (MAE) and root mean squared error (RMSE) in original units, with the sample standard deviation across those three fits. This variation reflects initialization on a fixed test period; it is not a confidence interval for performance on other time periods. Both smart-home temperature targets are reported separately, and the two imputed test targets per room are excluded from scoring.

Figure 2 divides each test RMSE by the corresponding persistence RMSE, making improvements over that baseline visible without combining incompatible physical units. All three RNNs improve on persistence in mean RMSE for minimum

temperatures, both smart-home targets, sunspots and AR(2). For air passengers, persistence outperforms every RNN on both metrics. The lowest mean RMSE among RNNs is achieved by Multi for air passengers and both smart-home targets, Elman for minimum temperatures and sunspots, and Jordan for AR(2). These are descriptive rankings; small differences, particularly for AR(2), do not establish a statistically significant advantage. Rankings also depend on the error measure: Jordan has the lowest RNN MAE for air passengers.

A. Interpretation by Dataset

1) *Air passengers*: Table IV provides the clearest example of why a simple baseline is necessary. For air passengers, Multi has the lowest RNN test RMSE, 76.290 thousand passengers, but its error is still 47.3% higher than persistence’s 51.782, as also shown by the passenger panel of Fig. 2. Elman, which ranked first during cross-validation, does not retain that ranking on the final period. Thus, the architecture selected as best on development folds need not remain best on later data.

The passenger panel in Fig. 3 shows that the seed-11 RNN predictions underestimate the two summer peaks. The development data reach a maximum of 505 thousand passengers, whereas the test data reach 622 thousand. This requires extrapolation to levels beyond those seen during fitting, in a series whose seasonal amplitude also grows. Persistence can track the changing level using the most recent observation, although it reacts one month late to changes. The RNNs’ peak underestimation is consistent with difficulty extrapolating this growth. It does not, by itself, establish whether limited capacity, optimization, or the chosen representation is responsible.

Only 120 monthly rows are available before the holdout, and Table III shows that the selected Elman and Jordan training durations are 99 epochs, close to the 100-epoch search limit. These facts limit any claim that training reached an ideal fit. Table IV shows that Jordan has lower MAE than the other RNNs but higher RMSE than Multi: its average absolute error is smaller while its larger errors receive a greater penalty under RMSE. A seasonal baseline, a trend-aware statistical model, or transformations selected on development data would provide useful additional comparisons. They were not tested here.

2) *Minimum temperatures*: For minimum temperatures, Table IV shows that Elman reduces test RMSE from 2.481 to

TABLE III

SETTINGS SELECTED USING DEVELOPMENT DATA. EACH CELL GIVES HIDDEN UNITS / TRAINING EPOCHS / TRAINABLE PARAMETERS. FINAL FITS USE THREE INITIALIZATION SEEDS AND THE SAME SELECTED SETTINGS.

Dataset	Elman	Jordan	Multi
Air passengers	16 / 99 / 305	8 / 99 / 33	32 / 69 / 1153
Minimum temperatures	8 / 36 / 89	32 / 11 / 129	8 / 29 / 97
Smart home (both targets)	32 / 60 / 1250	32 / 58 / 290	32 / 54 / 1314
Sunspots	16 / 16 / 305	16 / 48 / 65	32 / 13 / 1153
AR(2)	32 / 14 / 1121	16 / 40 / 65	32 / 19 / 1153

TABLE IV

FINAL TEST ERRORS IN ORIGINAL UNITS. RNN ENTRIES ARE MEAN $\pm$ SAMPLE STANDARD DEVIATION ACROSS INITIALIZATION SEEDS 11, 22 AND 33. PERSISTENCE IS DETERMINISTIC. BOLD INDICATES THE LOWEST UNROUNDED MEAN IN EACH ROW, INCLUDING PERSISTENCE; IT DOES NOT INDICATE STATISTICAL SIGNIFICANCE. N COUNTS OBSERVED TEST TARGETS.

Dataset / target	Metric	N	Elman	Jordan	Multi	Persistence
Air passengers	MAE	24	66.839 $\pm$ 10.259	60.587 $\pm$ 4.737	65.360 $\pm$ 3.292	44.208
	RMSE	24	80.303 $\pm$ 10.780	79.682 $\pm$ 6.765	76.290 $\pm$ 2.299	51.782
Minimum temperatures	MAE	730	1.742 $\pm$ 0.007	1.854 $\pm$ 0.016	1.747 $\pm$ 0.006	1.953
	RMSE	730	2.203 $\pm$ 0.004	2.346 $\pm$ 0.016	2.208 $\pm$ 0.005	2.481
Smart home: dining room	MAE	670	0.0336 $\pm$ 0.0146	0.0325 $\pm$ 0.0068	0.0255 $\pm$ 0.0023	0.1176
	RMSE	670	0.0421 $\pm$ 0.0156	0.0417 $\pm$ 0.0079	0.0332 $\pm$ 0.0019	0.1345
Smart home: bedroom	MAE	670	0.0305 $\pm$ 0.0013	0.0349 $\pm$ 0.0029	0.0287 $\pm$ 0.0038	0.1138
	RMSE	670	0.0395 $\pm$ 0.0019	0.0475 $\pm$ 0.0020	0.0391 $\pm$ 0.0047	0.1347
Sunspots	MAE	264	11.902 $\pm$ 0.400	12.097 $\pm$ 0.219	12.778 $\pm$ 0.354	12.595
	RMSE	264	16.793 $\pm$ 0.452	16.912 $\pm$ 0.167	17.515 $\pm$ 0.509	17.834
AR(2)	MAE	1000	0.8163 $\pm$ 0.0022	0.8140 $\pm$ 0.0037	0.8156 $\pm$ 0.0069	0.8699
	RMSE	1000	1.0193 $\pm$ 0.0019	1.0185 $\pm$ 0.0074	1.0195 $\pm$ 0.0060	1.0868

Units: passengers in thousands; temperatures in $^{\circ}\text{C}$; sunspots as an index; AR(2) in arbitrary units.

2.203 $^{\circ}\text{C}$, an 11.2% improvement over persistence. Multi is nearly identical at 2.208 $^{\circ}\text{C}$; their difference of approximately 0.004 $^{\circ}\text{C}$ is small compared with the overall forecast error. Jordan also improves on persistence, but less strongly. The minimum-temperature panel in Fig. 3 shows that the RNNs follow much of the local variation while smoothing some sharp changes. This is consistent with learning predictable short-term dependence while leaving substantial daily variation unexplained.

The selected Elman and Multi models both use eight hidden units (Table III). This shows that the smallest tested size was sufficient to give their best validation scores under this setup; it does not prove that every larger network overfit. Training curves and additional controlled comparisons would be needed to distinguish overfitting from optimization variability. Furthermore, the 30-day input window is shorter than an annual cycle. These results therefore do not demonstrate that the networks learned annual seasonality explicitly. Their small seed-to-seed standard deviations indicate repeatable fitting on this split, not guaranteed robustness to other climates or future periods.

3) *Smart-home temperatures*: Table IV shows that Multi obtains the lowest mean errors for both rooms. Its RMSE falls from the persistence baseline’s 0.1345 to 0.0332 $^{\circ}\text{C}$ for the dining room and from 0.1347 to 0.0391 $^{\circ}\text{C}$ for the bedroom: reductions of 75.3% and 70.9%, respectively, visible in the two smart-home panels of Fig. 2. Nevertheless, its bedroom RMSE

is only slightly below Elman’s 0.0395 $^{\circ}\text{C}$. The dining-room result shows a clearer numerical separation, while Elman’s larger standard deviation there indicates greater sensitivity to initialization in these three fits.

These large relative improvements are plausible for a smoothly changing temperature signal forecast only 15 minutes ahead. The models receive a day’s history of both indoor temperatures, outdoor temperature and outdoor humidity, whereas persistence uses only the latest target value. The two smart-home panels in Fig. 3 show smooth rises and falls that a model can potentially anticipate from recent changes. However, the experiment includes no input-feature ablation or simpler trend-extrapolation baseline, so it cannot determine whether the improvement comes from extra sensors, the longer history, or the recurrent architecture itself. All three RNNs have the same inputs, making their mutual comparison more controlled than their comparison with persistence.

The holdout covers approximately one week in the same monitored house. Accuracy on this short interval does not establish performance across seasons, other buildings, or thermostat interventions. In particular, a 15-minute forecast evaluated using newly observed inputs at every step is a substantially narrower task than predicting indoor temperature several hours ahead without further measurements.

4) *Sunspots*: For sunspots, Table IV gives Elman the lowest mean RMSE, 16.793, compared with 17.834 for persistence, a 5.8% reduction. Jordan is close at 16.912, and Multi is less

accurate at 17.515. The metric matters here too: although Multi improves on persistence in RMSE, its MAE is worse, 12.778 versus 12.595. A reduction in squared-error sensitivity to large misses does not necessarily improve the typical absolute error.

The 132-month input window (Table I) makes roughly one solar cycle available to each prediction. The sunspot panel of Fig. 3 illustrates the one-step predictions tracking much of the changing level while missing some abrupt month-to-month changes. However, the target remains the next month’s value, and persistence is already a strong comparator. The modest improvement does not establish that the networks can predict the timing or amplitude of an entire future solar cycle. The test period spans about two nominal cycles, which also limits evidence about performance across different long-term solar regimes.

5) *Synthetic AR(2)*: For AR(2), the three mean test RMSEs in Table IV lie between 1.0185 and 1.0195, compared with 1.0868 for persistence. Jordan’s numerical lead corresponds to a 6.3% improvement over persistence, but the separation between architectures is smaller than their individual seed-to-seed standard deviations. Cross-validation slightly favoured Multi instead. The nearly coincident markers in the AR(2) panel of Fig. 2 illustrate this small separation, while its forecast panel in Fig. 3 shows similar prediction trajectories for the displayed seed. These near-ties provide little evidence for a practically meaningful architecture advantage on this task.

This is consistent with the deliberately simple generating process: the conditional mean depends linearly on only the two supplied observations. With independent innovations of variance one, the known-coefficient conditional-mean predictor has population MSE one and hence population RMSE one. The RNN errors are close to this theoretical reference. However, it is not an exact lower bound for an empirical RMSE on a finite test sample, and the oracle’s realized test error has not been computed. Only one generated realization and one coefficient setting are used, so the result should not be generalized to arbitrary stationary processes or long-memory problems.

B. Scope and Limitations of the Comparison

The experiment compares complete training configurations rather than isolating feedback structure alone. Hidden sizes are selected separately for each architecture, and equal hidden sizes do not imply equal numbers of parameters. For example, Table III shows that the smart-home Jordan model has 290 trainable parameters, compared with 1,250 for Elman and 1,314 for Multi. Its competitive errors therefore represent a potentially useful accuracy-size trade-off for embedded applications. Parameter counts alone do not establish inference latency, energy consumption or memory requirements on a target device; no embedded deployment benchmark was performed.

The search is limited to hidden sizes 8, 16 and 32, one hidden layer, fixed input windows and a shared optimizer configuration. Cross-validation uses one initialization seed, while the final fits use three. Consequently, hyperparameter selection may itself depend on initialization, and the final standard deviations do not quantify that selection uncertainty. Sliding-window validation

is supported but was not evaluated. A different training window, forecast horizon or training budget could alter the rankings. For each model, the final epoch count is derived from inner early-stopping results; it is not selected using test loss and is not guaranteed to be optimal for the full development period.

The models reset recurrent context for every input window. The reported task is therefore prediction from a finite recent history, with actual observations becoming available before each subsequent forecast. It does not test persistent state across the entire series or recursive forecasting without new observations. Data quality also constrains interpretation: masking avoids scoring explicitly imputed targets, but forward-filled values can still occur in input histories, and retained source values may themselves be uncertain or provisional.

Finally, the test errors share timestamps and are temporally dependent, and the three seeds all use the same data split. No formal significance test was performed; neither a small difference in means nor overlap of seed standard deviations establishes significance or equivalence. The fixed-seed excerpts in Fig. 3 illustrate behaviour but cannot replace the full-period scores in Table IV. Additional chronological holdouts, more initialization seeds, stronger non-neural baselines and matched-parameter comparisons would strengthen future conclusions. Within the present design, no architecture is uniformly best: the relative benefit depends on the dataset, target and error measure, and persistence remains the strongest tested method for air passengers.

VI. CONCLUSION

This section draws together the main findings, their practical implications and directions for further work.

This study compared three recurrent neural network architectures across five time-series forecasting problems. The shared pipeline provided a consistent way to prepare data, select model settings and evaluate predictions on later observations. The experiments demonstrate that forecasting performance depends on the dataset and evaluation conditions, rather than on feedback structure alone.

No architecture was uniformly best. Although the networks improved on persistence in mean RMSE for four datasets, persistence performed better on the airline-passenger test period (Fig. 2). Several differences between architectures were small, so their numerical rankings should not be treated as conclusive evidence of superiority. These findings highlight the importance of simple baselines and careful interpretation of results.

The conclusions are limited to the tested configurations, data periods and one-step forecast horizon. Further work could examine stronger baselines, longer horizons and additional evaluation periods. For embedded applications, practical measurements of memory use, latency and energy would also be needed. The main lesson is that selecting a forecasting model requires considering accuracy, complexity and the intended use together.

REFERENCES

- [1] J. D. Kelleher, B. Mac Namee, and A. D’Arcy, *Fundamentals of Machine Learning for Predictive Data Analytics: Algorithms, Worked Examples, and Case Studies*, 2nd ed. MIT Press, 2020.

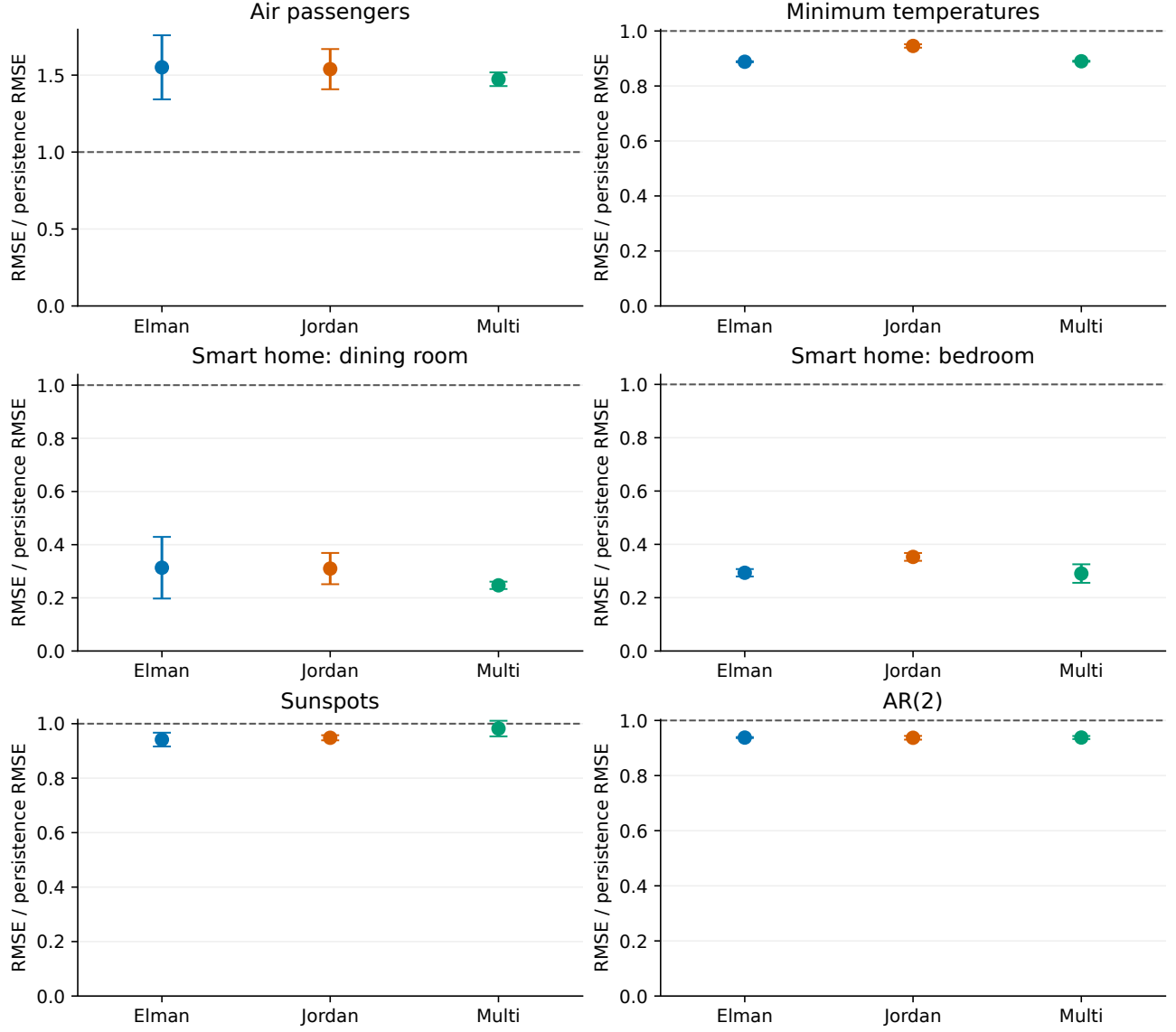


Fig. 2. Final test RMSE relative to persistence, with mean $\pm$ one sample standard deviation across three initialization seeds. Values below the dashed line indicate improvement over persistence. Panels have independent vertical scales and include both smart-home targets.

- [2] J. L. Elman, "Finding structure in time," *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/036402139090002E>
- [3] M. I. Jordan, "Serial order: A parallel distributed processing approach," *Advances in psychology*, vol. 121, pp. 471–495, 1997. [Online]. Available: <https://api.semanticscholar.org/CorpusID:15375627>
- [4] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, Y. W. Teh and M. Titterton, Eds., vol. 9. Chia Laguna Resort, Sardinia, Italy: PMLR, 13–15 May 2010, pp. 249–256. [Online]. Available: <https://proceedings.mlr.press/v9/glorot10a.html>
- [5] A. M. Saxe, J. L. McClelland, and S. Ganguli, "Exact solutions to the nonlinear dynamics of learning in deep linear neural networks," 2014. [Online]. Available: <https://arxiv.org/abs/1312.6120>
- [6] D. A. Dickey and W. A. Fuller, "Distribution of the estimators for autoregressive time series with a unit root," *Journal of the American Statistical Association*, vol. 74, no. 366, pp. 427–431, 1979.
- [7] D. Kwiatkowski, P. C. B. Phillips, P. Schmidt, and Y. Shin, "Testing the null hypothesis of stationarity against the alternative of a unit root," *Journal of Econometrics*, vol. 54, no. 1–3, pp. 159–178, 1992.
- [8] G. E. P. Box, G. M. Jenkins, and G. C. Reinsel, *Time Series Analysis: Forecasting and Control*, 3rd ed. Holden-Day, 1994.
- [9] F. Zamora-Martínez, P. Romeu-Guallart, and J. Pardo, "On-line learning of indoor temperature in a passive building using artificial neural networks," *Energy and Buildings*, vol. 72, pp. 303–316, 2014.
- [10] F. Clette, L. Svalgaard, J. M. Vaquero, and E. W. Cliver, "Revisiting the sunspot number: A 400-year perspective on the solar cycle," *Space Science Reviews*, vol. 186, pp. 35–103, 2014.
- [11] R. J. Hyndman and G. Athanasopoulos, "Forecasting: Principles and practice, section 9.3: Autoregressive models," <https://otexts.com/fpp3/AR.html>, 2021.

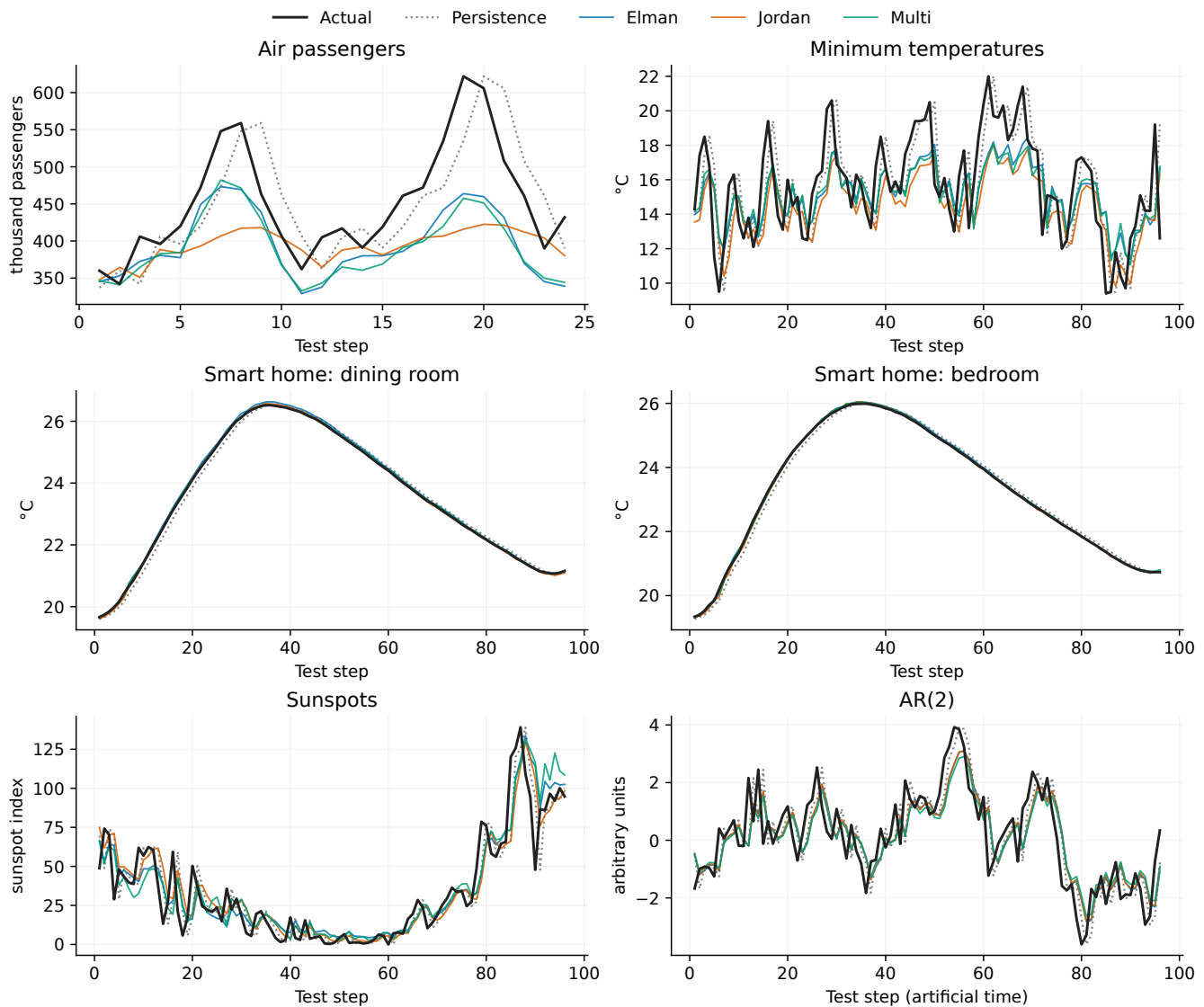


Fig. 3. One-step-ahead forecast examples using initialization seed 11 for every architecture. Each panel shows the first 96 test rows, or all 24 for air passengers, without selecting an interval based on model performance. Curves represent individual models, not an ensemble. Actual observations become available before subsequent forecasts. Full test periods, rather than these display excerpts, are used for all reported metrics. AR(2) time labels are artificial.